VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
MEHUL KHARE	24BAI10631
Slot	**Date of Submission**
B22	31-03-2026

Table of Contents

Section No.	Section Title & Sub-topics	Page No.
1.	**Part A-- Origin and Philosophy**	**3**
2.	**Part A2: The License — What it Actually Says**	4
3.	**Part A3: The Ethics of Open Source**	5
4.	**Part B: Linux Footprint**	6-7
5.	**Part C: The FOSS Ecosystem**	8
6.	**Part D: Comparative Analysis**	9
7.	**Shell Script Documentation**	10-14
8.	**Conclusion and Personal Reflections**	15
9.	**Project Deliverables**	16

Part A-- Origin and Philosophy

Part A1-- The Video LAN Origin Story

Most people know VLC as the player that "just works," but its roots are actually in a 1996 student project at **École Centrale Paris**. Back then, the campus network used "Token Ring" technology, which was incredibly slow and definitely not built for video. The students wanted to stream television to their dorms, but there was a massive bottleneck: proprietary software was either too expensive or simply couldn't handle the bandwidth.

Instead of waiting for a company to fix it, a small group of engineering students decided to build their own solution from scratch. They created the **VideoLAN project**, which was originally a two-part system: a server to broadcast the video and a client (VLC) to receive it. Because they were students learning as they went, they built it to be "modular"—meaning it could be easily adapted to different systems and video formats.

For a few years, it stayed a hidden gem inside the university. The real turning point came in **2001**. The project leads realized that the world needed a player that wasn't tied to corporate interests or expensive licenses. They fought to release the code under the **GPL (General Public License)**. This move turned VLC from a campus experiment into a global community effort.

The creators chose to share it rather than sell it because they believed in **academic freedom**. They knew that if they kept the code "closed," it would likely die after they graduated. By opening it up, they allowed thousands of developers worldwide to add "codecs" and features. This is why today, VLC can play almost any file type—it’s the result of 25 years of collective human effort rather than a single corporate product.

Part A2: The License — What it Actually Says

When I ran vlc --version on my machine, the terminal immediately flagged that VLC is distributed under the GNU General Public License (GPL). Reading the actual license text was an eye-opening exercise in digital civil rights. It isn't just a legal document; it’s a guarantee of the "Four Freedoms":

1. Freedom 0: The right to run the program for any purpose.
2. Freedom 1: The right to study how the program works and change it.
3. Freedom 2: The right to redistribute copies.
4. Freedom 3: The right to distribute copies of your modified versions to others.

A common point of confusion I had to clear up was the difference between "free as in free beer" and "free as in freedom". VLC is both, but the "freedom" part is more important. If VLC were just "free beer," it would be like a gift you can use but never look inside or repair. Because it is "free as in freedom," the community owns the "recipe" (the source code).

I also researched the difference between GPL and MIT licensing. The GPL is a "copyleft" license, meaning if a company modifies VLC and shares it, they must also share their changes under the same license. They cannot turn VLC into a "black box" and sell it as proprietary. If I were building a tool to change the world, I would choose the GPL because it ensures the software stays in the hands of the people forever.

Part A3: The Ethics of Open Source

Reflecting on the ethics of this model, I realized that open source is a powerful statement about how knowledge should be shared.

Should all software be open source?

There is a strong argument for "Yes". When code is open, it is safer because anyone can audit it for bugs or "backdoors". However, the "No" argument is also valid: programmers deserve to be paid for their labor, and proprietary models allow for massive R&D budgets that small communities might struggle to match.

Standing on the shoulders of giants

In software, this phrase means that no one starts from zero. VLC didn't invent video math; it built upon decades of research into compression and networking. By sharing their work, the VideoLAN team allows the next generation of students to build even better tools without "reinventing the wheel".

Part B: Linux Footprint

Installation and Package Management

To begin my audit, I had to move from a theoretical understanding to a practical one by installing VLC on my **Ubuntu 24.04 LTS** environment. I chose the standard **apt** (Advanced Package Tool) method because it is the most reliable way to manage dependencies in the Debian/Ubuntu ecosystem.

By running sudo apt update and sudo apt install vlc, I wasn't just downloading a player; I was integrating VLC into the system's shared library architecture. This method ensures that the open-source community can push security patches directly to my machine through regular system updates.

Locating the Files

Once installed, I used the Linux terminal to track where the software "lives".

- **Binaries:** Using the which vlc command, I found the executable located in /usr/bin/vlc. This is the standard directory for user-executable programs in the Linux Filesystem Hierarchy Standard (FHS).
- **Configuration:** I discovered that VLC stores its personal user settings in a hidden directory within my home folder: ~/.config/vlc. This separation of the program (in /usr) and the settings (in /home) is a classic Linux design that allows multiple users to have their own custom VLC experience on the same machine.

Security and User Permissions

One of the most important things I noticed is how VLC handles security. When I run the command ls -l /usr/bin/vlc, I can see that the file is owned by the **root** user, but it is designed to be executed by a **standard user** (like dashk).

This is a critical security layer: if a malicious video file tried to exploit a bug in VLC, the "damage" would be restricted to my user files and wouldn't be able to "break out" and delete system-level files. VLC actually warns you if you try to run it as the root user because it’s considered an unsafe practice in the open-source world.

Service and Process Management

VLC isn't just a window you click on; it can be managed through the command line.

- **Standard Launch:** Typing vlc starts the full interface.
- **Headless Mode:** Using cvlc allows the software to run as a background service without a graphical interface—perfect for a server environment.
- **Monitoring:** While running, I used the top command to monitor the process, noting that it efficiently manages CPU and memory usage, reflecting its lightweight, student-built heritage.

Part C: The FOSS Ecosystem

C1. Standing on the Shoulders of Giants

One thing I realized during this audit is that VLC isn’t a "lonely" application. It’s actually a masterpiece of integration. It doesn't try to do everything itself; instead, it relies on a massive backend engine called **FFmpeg**.

FFmpeg is the "invisible hero" of the open-source world. By using it, VLC can play almost any video format without needing expensive, proprietary licenses. This is a perfect example of the **FOSS Ecosystem** in action: specialized groups build high-quality "bricks" (like libraries), and projects like VLC assemble them into a "house" for the user.

C2. Community Governance

Unlike big tech companies with CEOs and shareholders, VLC is run by a non-profit called the **VideoLAN Organization**. My research showed that they operate as a "meritocracy." Decisions aren't made based on profit margins, but on who contributes the best code. This ensures the software evolves based on what we actually need, rather than what will generate the most ad revenue or data-mining opportunities.

Part D: Comparative Analysis

D1. The Transparency Gap

The biggest difference is what I call the "Transparency Gap." With VLC, if I'm worried about my privacy, I can literally go to their GitHub and check the source code myself. With Windows Media Player, it's a "black box"—I just have to trust a corporation's word. In an age of data leaks, having a "glass box" like VLC is a huge ethical advantage.

D2. Quick Comparison Table

Feature	VLC Media Player (FOSS)	Windows Media Player (Proprietary)
Licensing	GNU GPL (Full Freedom)	EULA (Many Restrictions)
Codec Support	Built-in & Community-run	Often requires extra downloads
Privacy	No tracking or telemetry	Linked to OS data collection
Portability	Runs on Linux, Mac, Android	Locked to Windows

D3. The Verdict

My conclusion is that while proprietary players are fine for basic use, they create a "walled garden" that limits the user. VLC is more like a "Universal Key." It belongs to the user, not a vendor. For any student or developer, choosing VLC isn't just about the $0 price tag—it's about digital sovereignty

Shell Script Documentation

Script 1: System Identity and Kernel Reporting

Source Code:

```
mehul_13@hp: /mnt/c/Users/k
  GNU nano 7.2                                                    script1.sh *
#!/bin/bash
# Script 1: System Identity Report
# Author: Mehul Khare
# Purpose: Display basic system details along with VLC info

# --- User Details ---
STUDENT_NAME="Mehul Khare"
SOFTWARE_CHOICE="VLC Media Player"

# --- System Information Commands ---
KERNEL=$(uname -r)              # Get kernel version
USER_NAME=$(whoami)             # Get current user
UPTIME=$(uptime -p)             # Get system uptime
DATE=$(date)                    # Current date & time
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d '"')  # Linux distro name

# --- Output Section ---
echo "================================"
echo " Open Source Audit - $STUDENT_NAME"
echo "================================"
echo "Software: $SOFTWARE_CHOICE"
echo "Distro  : $DISTRO"
echo "Kernel  : $KERNEL"
echo "User    : $USER_NAME"
echo "Uptime  : $UPTIME"
echo "Date    : $DATE"
echo "License : GPL / LGPL (VLC Media Player)"   # VLC license info
```

Output:

```
mehul_13@hp:/mnt/c/Users/khare$ nano script1.sh
mehul_13@hp:/mnt/c/Users/khare$ ./script1.sh
================================
 Open Source Audit - Mehul Khare
================================
Software: VLC Media Player
Distro  : Ubuntu 24.04.4 LTS
Kernel  : 6.6.87.2-microsoft-standard-WSL2
User    : mehul_13
Uptime  : up 17 minutes
Date    : Mon Mar 30 13:22:20 UTC 2026
License : GPL / LGPL (VLC Media Player)
mehul_13@hp:/mnt/c/Users/khare$
```

Technical Explanation: This script utilizes **command substitution**—using $(command)—to dynamically fetch system metadata. It demonstrates how Shell can interact directly with the Linux kernel to report environment details essential for a software audit.

Script 2: FOSS Package Inspector (VLC Audit)

Source Code:

```
mehul_13@hp: /mnt/c/Users/l
  GNU nano 7.2                                                    script2.sh *
#!/bin/bash
# Script 2: FOSS Package Inspector
# Purpose: Check if VLC is installed and show details

PACKAGE="vlc"   # Package name

# --- Check if package is installed ---
# dpkg -l lists installed packages, grep checks for vlc
if dpkg -l | grep -qw $PACKAGE; then
    echo "$PACKAGE is installed."

    # Show version and description
    dpkg -s $PACKAGE | grep -E 'Version|Maintainer|Description'
else
    echo "$PACKAGE is NOT installed."
fi

# --- Case Statement for description ---
case $PACKAGE in
    vlc) echo "VLC: open-source media player that plays almost any format" ;;
    firefox) echo "Firefox: privacy-focused browser" ;;
    mysql) echo "MySQL: database system" ;;
    apache2) echo "Apache: web server powering internet" ;;
    *) echo "Unknown package" ;;
esac
```

Output:

```
mehul_13@hp:/mnt/c/Users/khare$ nano script2.sh
mehul_13@hp:/mnt/c/Users/khare$ ./script2.sh
vlc is installed.
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Version: 3.0.20-3build6
Description: multimedia player and streamer
Original-Maintainer: Debian Multimedia Maintainers <debian-multimedia@lists.debian.org>
VLC: open-source media player that plays almost any format
mehul_13@hp:/mnt/c/Users/khare$
```

Technical Explanation: This script implements **conditional branching** using an if-else statement. It leverages the dpkg package manager and grep to search through installed software, automating the verification process required for Unit 2 of the project.

Script 3: Directory and Disk Permission Auditor

Source Code:

```
mehul_13@hp: /mnt/c/Users/
  GNU nano 7.2                                          script3.sh *
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Purpose: Check directory size, permissions, and VLC config

# List of important system directories
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

# --- Loop through directories ---
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Get permissions, owner, group
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')

        # Get directory size
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done

# --- VLC Config Directory Check ---
CONFIG="$HOME/.config/vlc"

if [ -d "$CONFIG" ]; then
    echo "VLC Config Found:"
    ls -ld $CONFIG   # Show permissions
else
    echo "VLC config directory not found"
fi
```

Output:

```
mehul_13@hp:/mnt/c/Users/khare$ nano script3.sh
mehul_13@hp:/mnt/c/Users/khare$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.4M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 17M
/home => Permissions: drwxr-xr-x root root | Size: 36K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 98M
/tmp => Permissions: drwxrwxrwt root root | Size: 24K
VLC config directory not found
mehul_13@hp:/mnt/c/Users/khare$
```

Technical Explanation: This script demonstrates the use of a **for loop** to iterate through an array of system paths. It combines the du (disk usage) and ls commands to provide a snapshot of the Linux filesystem hierarchy and its current security permissions.

Script 4: Automated Log File Analyzer

Source Code:

```
mehul_13@hp: /mnt/c/Users/
  GNU nano 7.2                                                    script4.sh *
#!/bin/bash
# Script 4: Log File Analyzer
# Purpose: Count keyword occurrences in log file

LOGFILE=$1                      # First argument = log file
KEYWORD=${2:-"error"}           # Default keyword = error
COUNT=0                         # Counter

# --- Check if file exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found!"
    exit 1
fi

# --- Read file line by line ---
while IFS= read -r LINE; do
    # Check if line contains keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # Increment counter
    fi
done < "$LOGFILE"

# --- Output result ---
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

# --- Show last 5 matching lines ---
echo "Last 5 matches:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output:

```
mehul_13@hp:/mnt/c/Users/khare$ nano script4.sh
mehul_13@hp:/mnt/c/Users/khare$ ./script4.sh /var/log/syslog
Keyword 'error' found 7 times in /var/log/syslog
Last 5 matches:
2026-03-30T13:03:13.580329+00:00 hp systemd[1]: apport-autoreport.path - Process error reports when automatic reporting is enabled (file watch) was skipped
because of an unmet condition check (ConditionPathExists=/var/lib/apport/autoreport).
2026-03-30T13:03:13.580332+00:00 hp systemd[1]: apport-autoreport.timer - Process error reports when automatic reporting is enabled (timer based) was skippe
d because of an unmet condition check (ConditionPathExists=/var/lib/apport/autoreport).
2026-03-30T13:03:13.912216+00:00 hp snapd[184]: helpers.go:190: error trying to compare the snap system key: system-key missing on disk
2026-03-30T13:03:15.680216+00:00 hp wsl-pro-service[187]: #033[33mWARNING#033[0m Could not ensure valid Landscape configuration: could not ensure valid Land
scape configuration: could not register distro to Landscape: could not enable Landscape: /usr/bin/landscape-config: error: exit status 1.
2026-03-30T13:15:08.479485+00:00 hp wsl-pro-service[187]: #033[33mWARNING#033[0m Exiting after could not ensure valid Landscape configuration: could not reg
ister distro to Landscape: could not enable Landscape: /usr/bin/landscape-config: error: exit status 1.
mehul_13@hp:/mnt/c/Users/khare$
```

Technical Explanation: This script utilizes a **while-read loop** to process a text file line-by-line. It also uses **positional parameters** ($1, $2), allowing the user to pass custom filenames and search terms from the command line, fulfilling the requirements for Unit 5.

Script 5: Open Source Manifesto Generator

Source Code:

```
mehul_13@hp: /mnt/c/Users/l
  GNU nano 7.2                                                    script5.sh *
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Purpose: Take user input and generate a manifesto file

echo "Answer the following questions:"

# --- User Inputs ---
read -p "1. Tool you use daily: " TOOL
read -p "2. Freedom means: " FREEDOM
read -p "3. What will you build: " BUILD

# --- Date and file name ---
DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# --- Write content to file ---
echo "On $DATE, I believe in open source." > $OUTPUT
echo "I use $TOOL daily, especially VLC for media." >> $OUTPUT
echo "Freedom means $FREEDOM to me." >> $OUTPUT
echo "I want to build $BUILD and share it freely." >> $OUTPUT

# --- Display output ---
echo "Manifesto saved in $OUTPUT"
cat $OUTPUT
```

Output:

```
mehul_13@hp:/mnt/c/Users/khare$ nano script5.sh
mehul_13@hp:/mnt/c/Users/khare$ ./script5.sh
Answer the following questions:
1. Tool you use daily: VLC
2. Freedom means: Control
3. What will you build: a free coding platform
Manifesto saved in manifesto_mehul_13.txt
On 30 March 2026, I believe in open source.
I use VLC daily, especially VLC for media.
Freedom means Control to me.
I want to build a free coding platform and share it freely.
mehul_13@hp:/mnt/c/Users/khare$
```

Technical Explanation: This script focuses on **interactive user input** via the read command and **file redirection**. It uses > to create/overwrite a file and >> to append data, showcasing how shell scripts can generate automated documentation.

Conclusion and Personal Reflections

Stepping into this audit, I initially viewed **VLC Media Player** as just another utility on my desktop. However, the deeper I dug into its origins at **École Centrale Paris**, the more I realized that VLC isn't just a player—it's a statement on how software should work. Seeing how a group of students in 1996 solved a massive networking bottleneck by building their own tool rather than waiting for a corporate solution was incredibly inspiring. It reminded me that the best engineering often comes from necessity and a desire to share knowledge.

The practical side of this project was where the "theory" of the classroom really met the "reality" of the terminal. Writing the five shell scripts forced me to stop thinking like a Windows user and start thinking like a Linux auditor. For instance, creating **Script 4 (The Log Analyzer)** taught me that on a Linux system, nothing is hidden; if something goes wrong, there is a trail of breadcrumbs in the /var/log directory waiting to be found. Similarly, **Script 2** showed me the efficiency of the apt package manager and how tightly VLC integrates with the underlying OS.

Ultimately, this project shifted my perspective on what "Free" really means. It’s not just about the zero-dollar price tag. It’s about the **four freedoms**—the right to open the hood, see how the engine works, and even tune it myself. Moving forward, I find myself much more conscious of the licenses behind the tools I use. This audit has proven to me that open-source software isn't just a hobbyist's alternative; it is a robust, ethical, and highly professional ecosystem that I am excited to explore further in my career.

Project Deliverables

Project Deliverables

- **Public GitHub Repository:**

  (https://github.com/kharemehul0-crypto/oss-audit---24BAI10631-.git)

- **Submission Files:** README.md, script1.sh, script2.sh, script3.sh, script4.sh, script5.sh.
- **Environment:** Ubuntu 24.04 LTS running on Windows Subsystem for Linux (WSL2).